# ROS 2 as Middleware for the Hybrid A* Planner

A Ground-Up Guide: From Concepts to Your Running Code

Anmol Sureka

March 2026

## Contents


**1 What Is ROS 2 and Why Do We Need It? 2**
- 1.1 The Problem . . . 2
- 1.2 Analogy: A Post Office . . . 2
- 1.3 What ROS 2 Does *Not* Do . . . 2

**2 Core Concepts 3**
- 2.1 Nodes: Independent Programs . . . 3
- 2.2 Messages: Structured Data . . . 3
- 2.3 Topics: Publish/Subscribe Channels . . . 4

**3 Actions: Request-Response for Long Tasks 6**
- 3.1 Action Server (in planner_node.py) . . . 6
- 3.2 Action Client (how to send a goal) . . . 7
- 3.3 Why Actions Instead of Topics? . . . 8

**4 Lifecycle Nodes: Controlled Startup 8**
- 4.1 Why? . . . 8
- 4.2 State Machine . . . 8
- 4.3 What Happens at Each Transition . . . 9
- 4.4 Transitioning from the Command Line . . . 9

**5 Quality of Service (QoS): Delivery Guarantees 10**
- 5.1 Reliability . . . 10
- 5.2 Durability . . . 10

**6 The Adapter Pattern: Connecting Algorithm to ROS 11**
- 6.1 Data Converter (converters.py) . . . 11
  - 6.1.1 Quaternion ↔ Yaw . . . 11
  - 6.1.2 PoseStamped → State . . . 12
  - 6.1.3 List of States → nav_msgs/Path . . . 12
- 6.2 Map Adapter (map_adapter.py) . . . 12
  - 6.2.1 OccupancyGrid Data Format . . . 13
  - 6.2.2 Cost Field Computation . . . 14

**7 Parameters: Configuration Without Recompiling** **14**
7.1 Declaring Parameters . . . 14
7.2 Reading Parameters . . . 14
7.3 The params.yaml File . . . 15

**8 The Complete Data Flow** **15**
8.1 Step 1: Map Arrives . . . 15
8.2 Step 2: Planner Receives Map . . . 16
8.3 Step 3: Planning Request . . . 16
8.4 Step 4: Result Returned . . . 17

**9 Concurrency: Callback Groups and Executors** **17**
9.1 Callback Groups . . . 17
9.2 MultiThreadedExecutor . . . 18
9.3 Thread Safety: The Lock . . . 18

**10 Package Structure and Build System** **18**
10.1 Why Two Packages? . . . 18
10.2 Workspace Layout . . . 19
10.3 package.xml: The Manifest . . . 19
10.4 setup.py: Python Entry Points . . . 20
10.5 Building . . . 20

**11 Launch Files: Starting Multiple Nodes** **20**
11.1 Key Elements . . . 21

**12 How Core Imports Work** **21**

**13 Putting It All Together: Step by Step** **22**
13.1 Build . . . 22
13.2 Launch . . . 22
13.3 Configure and Activate . . . 22
13.4 Plan . . . 23
13.5 Verify . . . 23
13.6 Useful Debugging Commands . . . 23

**14 Integration with Real Systems** **23**

**15 Summary of Key Concepts** **24**

# 1 What Is ROS 2 and Why Do We Need It?

## 1.1 The Problem

You already have a working Hybrid A* path planner written in pure Python (`src/hybrid_astar/`). It takes a map and two poses (start, goal), and returns a list of `State` objects representing the planned path.

But a real autonomous vehicle has many *separate* software modules:

- **Perception** — cameras, LiDAR, SLAM producing a map
- **Planning** — your Hybrid A* planner
- **Control** — steering and throttle commands
- **Localization** — estimating the vehicle's current pose

Each of these is typically its own program, possibly written in different languages, running on different computers. They need a way to *talk* to each other: perception sends the map to planning, planning sends the path to control, etc.

**Key Idea**

**ROS 2** (Robot Operating System 2) is *middleware* — a communication framework that lets independent programs exchange data using a standardized protocol. It is *not* an operating system, and it does *not* contain your algorithm. Your algorithm lives in pure Python; ROS 2 just connects it to the outside world.

## 1.2 Analogy: A Post Office

Think of ROS 2 as a post office:

- Each program is a **house** (called a *node*).
- Houses communicate by sending **letters** (called *messages*).
- Letters are sent to a named **mailbox** (called a *topic*).
- Any house can **drop off** letters at a mailbox (*publish*) and any house can **pick up** letters from a mailbox (*subscribe*).
- The post office (ROS 2) handles delivery — you never directly connect to the other house.

For example, your SLAM module publishes a map on the `/map` mailbox. Your planner subscribes to `/map` and reads whatever map arrives. Neither module needs to know the other exists.

## 1.3 What ROS 2 Does *Not* Do

ROS 2 does not:

- Implement path planning algorithms
- Require a simulator (Gazebo, etc.)
- Require Nav2 or any specific framework
- Dictate how you write your algorithm code

In your project, the **entire** Hybrid A* algorithm — bicycle kinematics, Reeds-Shepp curves, heuristic calculation, A* search — is in `src/hybrid_astar/` with **zero** ROS imports. The ROS 2 package in `ros2_ws/src/av_planner/` is a thin wrapper that receives data from other modules, feeds it into the algorithm, and publishes the result.

# 2 Core Concepts

## 2.1 Nodes: Independent Programs

A **node** is a single executable program registered with ROS 2. Each node has a name and can create publishers, subscribers, services, and action servers.

In your project there are two nodes:

| Node | File | Purpose |
|---|---|---|
| `planner_node` | `nodes/planner_node.py` | Wraps Hybrid A*. Subscribes to `/map`, serves `/plan_path` action, publishes `/planned_path`. |
| `mock_map_node` | `nodes/mock_map_node.py` | Loads a PNG and publishes it as an `OccupancyGrid` on `/map` (for testing). |

A node is created by subclassing a ROS 2 base class:

```python
import rclpy
from rclpy.node import Node

class MockMapNode(Node):
    def __init__(self):
        super().__init__('mock_map_node')  # node name
        # ... setup publishers, subscribers, etc.

def main():
    rclpy.init()                # Initialize ROS 2
    node = MockMapNode()
    rclpy.spin(node)            # Keep running, processing callbacks
    node.destroy_node()
    rclpy.shutdown()
```

Listing 1: Creating a basic node (mock_map_node.py)

`rclpy.init()` starts the ROS 2 runtime. `rclpy.spin(node)` is an event loop — it keeps the node alive and dispatches incoming messages to your callback functions. Think of it like Python's `asyncio` event loop.

## 2.2 Messages: Structured Data

A **message** is a structured data type — like a Python dataclass or a C struct — that defines what information travels between nodes.

ROS 2 provides many standard message types:

| Message Type | Fields |
|---|---|
| `geometry_msgs/PoseStamped` | Header + position $(x, y, z)$ + orientation (quaternion $x, y, z, w$) |
| `nav_msgs/Path` | Header + list of `PoseStamped` |
| `nav_msgs/OccupancyGrid` | Header + metadata (width, height, resolution, origin) + flat array of cell values |
| `std_msgs/Header` | timestamp + frame ID (coordinate frame name) |

You also define your own. In your project, `PlanningStats.msg` carries planning metrics:

```
std_msgs/Header header
float64 search_time        # seconds
float64 path_length         # meters
int32   nodes_expanded      # how many nodes A* expanded
bool    analytical_expansion  # did Reeds-Shepp shot succeed?
```

Listing 2: msg/PlanningStats.msg

In Python, you use it like a normal object:

```python
from av_planner_interfaces.msg import PlanningStats

stats = PlanningStats()
stats.search_time = 0.188
stats.path_length = 80.3
stats.nodes_expanded = 27
stats.analytical_expansion = True
```

Listing 3: Using a custom message

**How custom messages get built**

You write `.msg` files in the `av_planner_interfaces` package. The ROS 2 build tool (`colcon build`) runs a code generator (`rosidl`) that converts these into importable Python classes. That is why you need a separate CMake package for interfaces — the generator requires CMake, even though your nodes are Python.

## 2.3 Topics: Publish/Subscribe Channels

A **topic** is a named channel. Any node can *publish* messages to a topic, and any node can *subscribe* to receive those messages. The connection is many-to-many and anonymous.

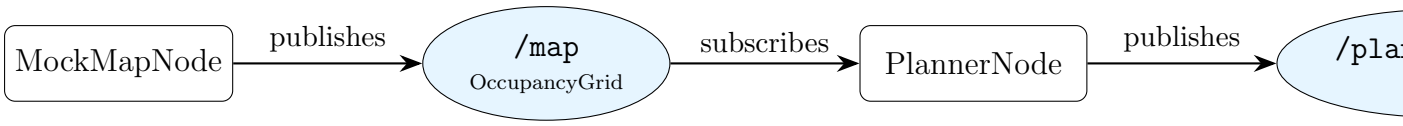


Figure 1: Data flow through topics in your system.

Here is how `mock_map_node.py` publishes the map:

```python
from nav_msgs.msg import OccupancyGrid

# In __init__:
self._pub = self.create_publisher(OccupancyGrid, '/map', qos)

# To publish:
msg = OccupancyGrid()
msg.info.width = 200
msg.info.height = 200
msg.info.resolution = 0.5
msg.data = grid.tolist()  # flat list of occupancy values
self._pub.publish(msg)
```

Listing 4: Publishing a message (mock_map_node.py, simplified)

And here is how `planner_node.py` subscribes to it:

```python
# In on_configure:
self._map_sub = self.create_subscription(
    OccupancyGrid,     # message type
    '/map',             # topic name
    self._on_map,       # callback function
    map_qos             # Quality of Service profile
)

def _on_map(self, msg: OccupancyGrid):
    """Called automatically whenever a new map is published."""
    self.get_logger().info(
        f'Map received: {msg.info.width}x{msg.info.height}'
    )
    # Convert OccupancyGrid data to our internal format
    self._build_planner(msg)
```

Listing 5: Subscribing to a topic (planner_node.py, simplified)

**Analogy**

A topic is like a radio frequency. The mock map node *broadcasts* on frequency `/map`. The planner node *tunes in* to that frequency. If you later replace the mock with a real SLAM system, you just make sure it broadcasts on the same frequency — the planner doesn't need to change at all.

Your project has three topics:

| Topic | Type | Publisher | Subscriber |
|---|---|---|---|
| `/map` | `OccupancyGrid` | mock_map_node (or SLAM) | planner_node |
| `/planned_path` | `Path` | planner_node | controller (future) |
| `/planning_stats` | `PlanningStats` | planner_node | monitor (future) |

# 3 Actions: Request-Response for Long Tasks

Topics are great for continuous data streams (like a map or sensor feed), but planning is a *one-shot, long-running* task: you send a start and goal, wait for the planner to compute, and get a path back. Topics don't naturally support this request-response pattern.

ROS 2 provides **actions** for this. An action has three parts:

1. **Goal** — what the client sends (the request)
2. **Feedback** — periodic updates while work is in progress
3. **Result** — what the server returns when done

Your `PlanPath.action` defines all three:

```
# Goal: what the client sends
geometry_msgs/PoseStamped start
geometry_msgs/PoseStamped goal
---
# Result: what the server returns
nav_msgs/Path path
PlanningStats stats
bool success
string message
---
# Feedback: updates during planning
string status
```

Listing 6: action/PlanPath.action

**Analogy: Ordering Food**

An action is like ordering at a restaurant:

- **Goal**: "I'd like a pizza with mushrooms" (start + goal poses)
- **Feedback**: "Your order is being prepared" (status = "planning")
- **Result**: The pizza arrives (the planned path + statistics)

You can also *cancel* the order (cancel the planning request).

## 3.1 Action Server (in planner_node.py)

The planner node creates an **action server** that waits for planning requests:

```python
from rclpy.action import ActionServer
from av_planner_interfaces.action import PlanPath

# In on_configure:
self._action_server = ActionServer(
    self,                # the node
    PlanPath,            # action type (auto-generated from .action file)
    '/plan_path',        # action name
    self._execute_plan # callback: what to do when a goal arrives
)
```

Listing 7: Creating an action server (planner_node.py)

When a client sends a goal, ROS 2 calls `_execute_plan`:

```python
def _execute_plan(self, goal_handle):
    # 1. Extract start and goal from the request
    start = pose_to_state(goal_handle.request.start, CoreState)
    goal  = pose_to_state(goal_handle.request.goal, CoreState)

    # 2. Publish feedback
    feedback = PlanPath.Feedback()
    feedback.status = 'planning'
    goal_handle.publish_feedback(feedback)

    # 3. Run the actual Hybrid A* algorithm (pure Python, no ROS)
    plan_result = self._planner.plan(start, goal)

    # 4. Convert result to ROS messages and return
    result = PlanPath.Result()
    if plan_result is not None:
        path, info = plan_result
        result.path = states_to_path(path, 'map', stamp)
        result.success = True
        goal_handle.succeed()
    else:
        result.success = False
        goal_handle.abort()
    return result
```

Listing 8: Action execute callback (planner_node.py, simplified)

## 3.2 Action Client (how to send a goal)

From the command line:

```
ros2 action send_goal /plan_path \
    av_planner_interfaces/action/PlanPath \
    "{start: {pose: {position: {x: 10.0, y: 50.0}, \
            orientation: {w: 1.0}}}, \
      goal: {pose: {position: {x: 90.0, y: 50.0}, \
            orientation: {w: 1.0}}}}"
```

Listing 9: Sending a planning goal via CLI

Or from another Python node:

```python
from rclpy.action import ActionClient
from av_planner_interfaces.action import PlanPath

client = ActionClient(self, PlanPath, '/plan_path')
client.wait_for_server()

goal = PlanPath.Goal()
goal.start.pose.position.x = 10.0
goal.start.pose.position.y = 50.0
goal.start.pose.orientation.w = 1.0
```

```
goal.goal.pose.position.x = 90.0
goal.goal.pose.position.y = 50.0
goal.goal.pose.orientation.w = 1.0

future = client.send_goal_async(goal)
```

Listing 10: Sending a goal from Python (future mission planner)

### 3.3 Why Actions Instead of Topics?

| | Topic | Action |
|---|---|---|
| Pattern | Continuous stream | Request → response |
| Blocking? | No (fire-and-forget) | Asynchronous (can await result) |
| Feedback | No | Yes (periodic updates) |
| Cancellation | No | Yes |
| Use case | Map, sensor data | Planning, navigation goals |

Planning is a perfect fit for actions because:

- It takes seconds (too long for a synchronous service call)
- The caller needs to know when it's done and whether it succeeded
- You might want to cancel a planning request mid-computation

## 4 Lifecycle Nodes: Controlled Startup

A regular node starts doing everything the moment it launches. A **lifecycle node** has explicit states and transitions, giving you control over when things happen.

### 4.1 Why?

Consider this scenario: the planner node starts, but the map hasn't arrived yet. With a regular node, you'd need manual checks everywhere ("is the map ready? is the planner initialized?"). With a lifecycle node, you keep the node in "inactive" state until everything is ready.

### 4.2 State Machine

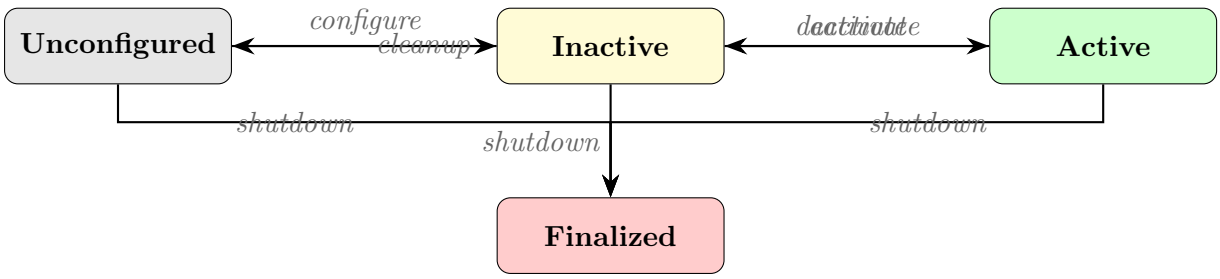


Figure 2: Lifecycle node state machine.

## 4.3 What Happens at Each Transition

In `planner_node.py`, each transition has a callback:

| Transition | What planner_node does |
|---|---|
| `__init__` (constructor) | Declare parameters only. No publishers, no subscribers, nothing heavy. |
| `on_configure` | Import core algorithm classes, create the map subscriber, action server, and publishers. Node is now *ready* but not processing. |
| `on_activate` | Log "Activated. Ready for planning requests." Node starts accepting action goals. |
| `on_deactivate` | Log "Deactivated." Temporarily stops accepting requests. |
| `on_cleanup` | Destroy all subscribers, publishers, action server. Release the planner. |

```python
from rclpy.lifecycle import Node as LifecycleNode,
    TransitionCallbackReturn

class PlannerNode(LifecycleNode):
    def __init__(self):
        super().__init__('planner_node')
        self._declare_params()          # Only declare parameters
        self._planner = None

    def on_configure(self, state):
        # Heavy setup: imports, subscribers, publishers, action
            server
        self._core_classes = _setup_core_imports(project_root)
        self._map_sub = self.create_subscription(...)
        self._action_server = ActionServer(...)
        self._path_pub = self.create_publisher(...)
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state):
        self.get_logger().info('Ready for planning requests.')
        return super().on_activate(state)

    def on_cleanup(self, state):
        self.destroy_subscription(self._map_sub)
        self._action_server.destroy()
        self._planner = None
        return TransitionCallbackReturn.SUCCESS
```

Listing 11: Lifecycle callbacks (planner_node.py)

## 4.4 Transitioning from the Command Line

After launching the node, you manually bring it to the active state:

```
# Node starts in "unconfigured" state
ros2 lifecycle set /planner_node configure
# Now it creates subscribers, publishers, action server
# and starts listening for /map

ros2 lifecycle set /planner_node activate
# Now it accepts planning requests via /plan_path action
```

Listing 12: Lifecycle transitions

# 5 Quality of Service (QoS): Delivery Guarantees

QoS profiles control *how* messages are delivered. The two most important settings for your project:

## 5.1 Reliability

- **RELIABLE**: Every message is guaranteed to arrive (retransmission if needed). Used for map and path — you cannot afford to miss these.
- **BEST_EFFORT**: Messages may be dropped. Used for high-frequency sensor data where losing one frame is okay.

## 5.2 Durability

- **VOLATILE**: Only subscribers connected at the moment of publishing receive the message.
- **TRANSIENT_LOCAL**: The publisher keeps the last message. New subscribers that connect *later* still receive it.

**Why TRANSIENT_LOCAL for /map?**

The mock map node publishes the map once. If `planner_node` starts *after* the map was published, it would miss it with VOLATILE durability. TRANSIENT_LOCAL ensures the planner receives the last published map even if it subscribes late. This is critical because the map is published infrequently (once, then heartbeats every 10 seconds).

```
from rclpy.qos import QoSProfile, ReliabilityPolicy,
    DurabilityPolicy

map_qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,        # don't drop
    durability=DurabilityPolicy.TRANSIENT_LOCAL,    # late joiners
        get it
    history=HistoryPolicy.KEEP_LAST,
    depth=1                                         # keep only
        latest map
)
```

Listing 13: QoS profile for map subscription (planner_node.py)

| Topic | Reliability | Durability | Why |
|---|---|---|---|
| `/map` | RELIABLE | TRANSIENT_LOCAL | Map is critical, published rarely |
| `/planned_path` | RELIABLE | VOLATILE | Path consumed immediately |
| `/planning_stats` | RELIABLE | VOLATILE | Stats consumed immediately |

**Important rule**: Publisher and subscriber QoS must be *compatible*. If the publisher is TRANSIENT_LOCAL, the subscriber must also be TRANSIENT_LOCAL. Mismatched QoS is the #1 cause of "my subscriber doesn't receive anything."

# 6 The Adapter Pattern: Connecting Algorithm to ROS

This is the central design pattern of your ROS 2 package. The core algorithm expects certain Python objects (`State`, `MapHandler`). ROS 2 provides different objects (`PoseStamped`, `OccupancyGrid`). The **adapters** translate between them.

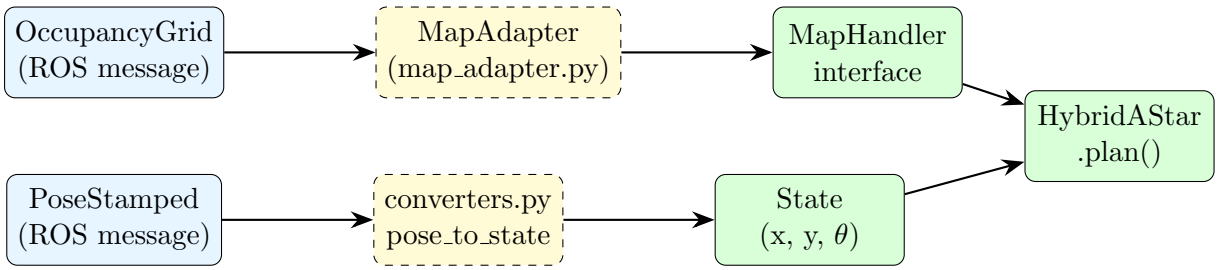


Figure 3: Adapter pattern: ROS messages → adapters → algorithm objects.

## 6.1 Data Converter (converters.py)

The converter functions translate between ROS 2 message types and the algorithm's `State` class. They contain **zero** algorithm logic — just data transformation.

### 6.1.1 Quaternion ↔ Yaw

ROS 2 represents orientation as a **quaternion** $(x, y, z, w)$ — a 4D rotation representation. Your algorithm uses a single angle $\theta$ (yaw = rotation about the Z axis). The conversion is:

$$\theta = \operatorname{atan2}\big(2(wz + xy),\ 1 - 2(y^2 + z^2)\big) \tag{1}$$

For 2D (no pitch or roll), $x = y = 0$, so this simplifies to $z = \sin(\theta/2)$, $w = \cos(\theta/2)$.

```python
def yaw_from_quaternion(q):
    siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
    cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
    return math.atan2(siny_cosp, cosy_cosp)

def quaternion_from_yaw(yaw):
    q = Quaternion()
    q.z = math.sin(yaw / 2.0)
    q.w = math.cos(yaw / 2.0)
    return q
```

Listing 14: Quaternion conversion (converters.py)

### 6.1.2 PoseStamped → State

```python
def pose_to_state(msg: PoseStamped, state_cls):
    x = msg.pose.position.x          # meters
    y = msg.pose.position.y          # meters
    theta = yaw_from_quaternion(msg.pose.orientation)  # radians
    return state_cls(x, y, theta)    # State(x, y, theta)
```

Listing 15: Converting a ROS pose to algorithm State

Note that `state_cls` is passed as a parameter — the converter file *never* imports from `src/`. This keeps the adapter layer free from any dependency on the algorithm code.

### 6.1.3 List of States → nav_msgs/Path

After planning, the algorithm returns a list of `State` objects. We convert each to a `PoseStamped` and pack them into a `Path`:

```python
def states_to_path(states, frame_id, stamp):
    path_msg = Path()
    path_msg.header.frame_id = frame_id  # "map"
    path_msg.header.stamp = stamp
    for s in states:
        path_msg.poses.append(state_to_pose(s, frame_id, stamp))
    return path_msg
```

Listing 16: Converting algorithm output to a ROS Path

## 6.2 Map Adapter (map_adapter.py)

This is the most important adapter. Your Hybrid A* algorithm expects a `MapHandler` object (from `src/map_handler.py`) which provides:

- `map_data` — 2D numpy array (0=free, 1=obstacle)
- `width`, `height` — map dimensions in meters
- `resolution` — meters per cell
- `cost_field` — distance-based cost field
- `world_to_pixel(x, y)` — coordinate conversion
- `is_obstacle_world(x, y)` — collision check

- `is_collision(points)` — check multiple points
- etc.

In the standalone Python planner, `MapHandler` loads a PNG image. In ROS 2, maps arrive as `OccupancyGrid` messages. The **MapAdapter** creates an object with the *same interface* as `MapHandler` but initializes from ROS data instead of a PNG file.

```python
class MapAdapter:
    def __init__(self, data, resolution, width, height,
                 origin_x=0.0, origin_y=0.0, obstacle_threshold=50):
        self.resolution = resolution
        self.pixel_width = width
        self.pixel_height = height

        # Convert OccupancyGrid values to binary:
        #   OccupancyGrid: -1=unknown, 0=free, 100=occupied
        #   MapHandler:    0=free, 1=obstacle
        grid = np.array(data, dtype=np.int8).reshape((height, width))
        self.map_data = np.where(
            (grid >= obstacle_threshold) | (grid < 0), 1, 0
        ).astype(np.uint8)

        self.width  = width * resolution    # meters
        self.height = height * resolution   # meters

        # Compute the same cost field as MapHandler
        self._compute_cost_field()
```

Listing 17: MapAdapter constructor (map_adapter.py, simplified)

**Duck Typing**

Python uses **duck typing**: "if it walks like a duck and quacks like a duck, it's a duck." `MapAdapter` doesn't inherit from `MapHandler`. It just has the same methods and attributes. When `HybridAStar` calls `map_handler.is_collision(points)`, Python doesn't check the type — it just looks for a method named `is_collision`. Since `MapAdapter` has one, it works.

### 6.2.1 OccupancyGrid Data Format

The OccupancyGrid uses a different value convention than your PNG maps:

| Source | Free | Occupied |
|---|---|---|
| PNG image | 255 (white) | 0 (black) |
| OccupancyGrid | 0 | 100 |
| MapHandler internal | 0 | 1 |

The MapAdapter handles this conversion. It also treats "unknown" cells ($-1$) as obstacles for safety.

### 6.2.2 Cost Field Computation

Both `MapHandler` and `MapAdapter` compute a cost field using **Euclidean distance transform**. For each free cell, the cost is inversely proportional to its distance from the nearest obstacle. Cells far from obstacles have low cost; cells near obstacles have high cost. This encourages the planner to prefer paths away from walls.

```
from scipy.ndimage import distance_transform_edt

def _compute_cost_field(self):
    free_space = (1 - self.map_data).astype(np.float64)
    self.distance_field = distance_transform_edt(free_space)
    max_dist = self.distance_field.max()
    if max_dist > 0:
        self.cost_field = 1.0 - np.clip(
            self.distance_field / max_dist, 0.0, 1.0
        )
    else:
        self.cost_field = np.zeros_like(self.map_data, dtype=np.
           float64)
```

Listing 18: Cost field computation (map_adapter.py)

# 7 Parameters: Configuration Without Recompiling

ROS 2 nodes can declare **parameters** — named values that can be set from YAML files, command line, or changed at runtime. This replaces hardcoded constants.

## 7.1 Declaring Parameters

In the constructor, the planner declares every configurable value:

```
def _declare_params(self):
    self.declare_parameter('frame_id', 'map')         # string
    self.declare_parameter('vehicle.wheel_base', 2.5) # float
    self.declare_parameter('motion.max_steering_angle', 40.0)
    self.declare_parameter('search.xy_resolution', 1.0)
    self.declare_parameter('cost.steering_penalty', 1.5)
    # ... etc.
```

Listing 19: Parameter declaration (planner_node.py)

The second argument is the default value. The actual value comes from `params.yaml` at launch time.

## 7.2 Reading Parameters

When building the planner, each parameter is read and passed to the algorithm constructor:

```
motion_model = MotionModel(
    wheel_base=self.get_parameter('vehicle.wheel_base').value,
```

```
    max_steering_angle=self.get_parameter('motion.max_steering_angle').value,
    step_size=self.get_parameter('motion.step_size').value,
)
```

Listing 20: Reading parameters to construct planner

### 7.3 The params.yaml File

All parameter values live in a single YAML file loaded at launch:

```yaml
planner_node:
  ros__parameters:
    frame_id: "map"
    project_root: "/path/to/hybrid_astar_planner"

    vehicle:
      wheel_base: 2.5        # meters
      length: 4.5
      width: 2.0

    motion:
      max_steering_angle: 40.0    # degrees
      step_size: 2.0              # meters
      num_steering_angles: 5

    cost:
      steering_penalty: 1.5
      reversing_penalty: 2.0
      direction_switch_penalty: 10.0

    heuristic:
      type: "max"     # h = max(RS distance, 2D Dijkstra)
      turning_radius: 2.98
```

Listing 21: config/params.yaml (excerpt)

The YAML nesting (e.g., `vehicle: {wheel_base: 2.5}`) becomes a dotted parameter name (`vehicle.wheel_base`). The top-level key `planner_node:` matches the node name, and `ros__parameters:` (with double underscores) is the required ROS 2 prefix.

## 8 The Complete Data Flow

Now let's trace a complete planning request through the system, from map arrival to path publication.

### 8.1 Step 1: Map Arrives

1. `mock_map_node` loads a PNG image (`maps/map_open.png`)
2. It converts the image to an `OccupancyGrid` message (flips vertically, thresholds pixels to 0/100)

3. It publishes the message on `/map` with TRANSIENT_LOCAL QoS

## 8.2 Step 2: Planner Receives Map

1. `planner_node`'s `_on_map` callback fires
2. It creates a `MapAdapter` from the OccupancyGrid data
3. It builds the *entire* planner pipeline:
   - `MapAdapter` → `MotionModel` → `VehicleFootprint` → `HeuristicCalculator` → `HybridAStar`
4. Stores the planner in `self._planner`

```python
def _build_planner(self, msg: OccupancyGrid):
    # Create map adapter (replaces MapHandler's PNG loading)
    self._map_handler = MapAdapter(
        data=np.array(msg.data, dtype=np.int8),
        resolution=msg.info.resolution,
        width=msg.info.width,
        height=msg.info.height,
    )

    # Build motion model (bicycle kinematics)
    motion_model = MotionModel(wheel_base=..., ...)

    # Build heuristic: h = max(RS distance, 2D Dijkstra)
    heuristic = HeuristicCalculator(
        map_handler=self._map_handler,  # MapAdapter works here!
        heuristic_type='max',
    )

    # Build planner: f = g + h, with cost penalties
    self._planner = HybridAStar(
        map_handler=self._map_handler,
        motion_model=motion_model,
        heuristic_calculator=heuristic,
        steering_penalty=1.5,
        reversing_penalty=2.0,
        # ...
    )
```

Listing 22: Planner pipeline construction (planner_node.py)

## 8.3 Step 3: Planning Request

1. A client sends a `PlanPath` goal with start and goal poses
2. `_execute_plan` callback fires
3. Converts `PoseStamped` → `State` using `pose_to_state`
4. Calls `self._planner.plan(start, goal)` — pure Python, no ROS
5. The algorithm runs: bicycle kinematics, A* search, Reeds-Shepp shots
6. Returns `(path, info)` where `path` is a list of `State`

### 8.4 Step 4: Result Returned

1. Converts `List[State]` → `nav_msgs/Path` using `states_to_path`
2. Fills `PlanningStats` from the info dict
3. Publishes path on `/planned_path` and stats on `/planning_stats`
4. Returns `PlanPath.Result` to the action client

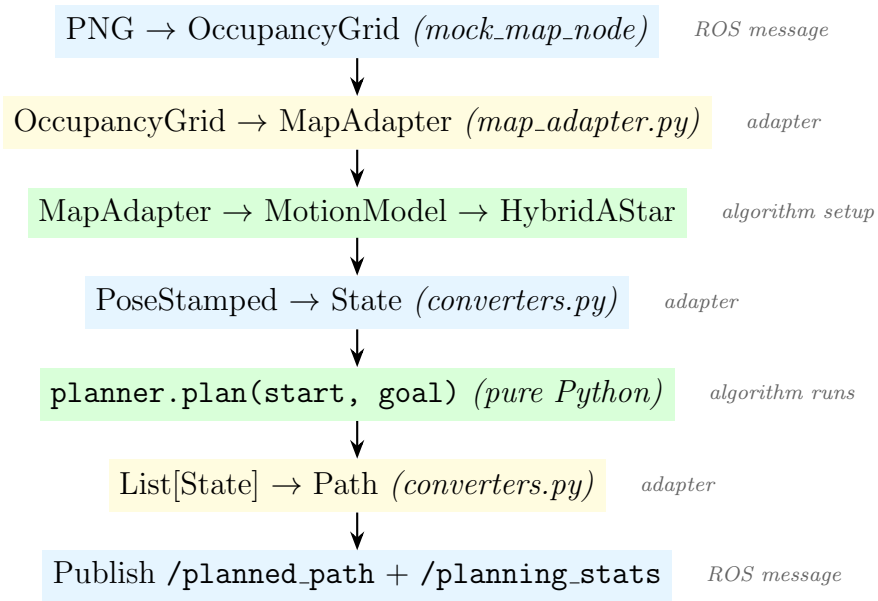


Figure 4: Complete data flow: from PNG to published path.

## 9 Concurrency: Callback Groups and Executors

By default, ROS 2 processes callbacks *one at a time* (single-threaded). This means if the planner is computing a path (which takes seconds), an incoming map update would have to wait. Your node solves this with two mechanisms.

### 9.1 Callback Groups

A **callback group** defines which callbacks can run concurrently:

```
self._map_cb_group = MutuallyExclusiveCallbackGroup()
self._action_cb_group = MutuallyExclusiveCallbackGroup()

self._map_sub = self.create_subscription(
    OccupancyGrid, '/map', self._on_map, map_qos,
    callback_group=self._map_cb_group     # group 1
)
self._action_server = ActionServer(
    self, PlanPath, '/plan_path', self._execute_plan,
    callback_group=self._action_cb_group  # group 2
)
```

Listing 23: Callback groups (planner_node.py)

Callbacks in *different* groups can run at the same time. Callbacks in the *same* group never overlap (mutually exclusive).

This means: a map update and a planning request can run concurrently, but two map updates cannot (which is what we want).

### 9.2 MultiThreadedExecutor

To actually run callbacks concurrently, we need multiple threads:

```
from rclpy.executors import MultiThreadedExecutor

executor = MultiThreadedExecutor(num_threads=2)
executor.add_node(node)
executor.spin()  # dispatches callbacks to threads
```

Listing 24: Multi-threaded executor (planner_node.py)

### 9.3 Thread Safety: The Lock

Since the map callback and planning callback can run simultaneously, we need a lock to prevent them from accessing the planner at the same time:

```
self._plan_lock = threading.Lock()

def _on_map(self, msg):
    with self._plan_lock:        # acquire lock
        self._build_planner(msg) # rebuild planner

def _execute_plan(self, goal_handle):
    with self._plan_lock:        # acquire same lock
        result = self._planner.plan(start, goal)
```

Listing 25: Thread safety with a lock

This ensures the planner is never being used while it's being rebuilt.

## 10 Package Structure and Build System

### 10.1 Why Two Packages?

| Package | Build Type | Contents |
|---|---|---|
| `av_planner_interfaces` | CMake | Custom message (`.msg`) and action (`.action`) definitions. CMake is required because the ROS 2 code generator (`rosidl`) needs CMake to generate Python bindings from `.msg`/`.action` files. |
| `av_planner` | Python (`ament_python`) | Your actual nodes, adapters, launch file, config. Pure Python. |

## 10.2 Workspace Layout

```
ros2_ws/
  src/
    av_planner_interfaces/     # Package 1: interfaces (CMake)
      package.xml
      CMakeLists.txt
      msg/PlanningStats.msg
      action/PlanPath.action

    av_planner/                # Package 2: nodes (Python)
      package.xml
      setup.py
      setup.cfg
      resource/av_planner      # empty marker (ament requirement)
      config/params.yaml
      launch/planner.launch.xml
      av_planner/              # Python module
        __init__.py
        adapters/
          __init__.py
          map_adapter.py
          converters.py
        nodes/
          __init__.py
          planner_node.py
          mock_map_node.py
```

Listing 26: Directory structure

## 10.3 package.xml: The Manifest

Every ROS 2 package has a `package.xml` that declares its name, dependencies, and build type:

```
<package format="3">
  <name>av_planner</name>
  <buildtool_depend>ament_python</buildtool_depend>

  <depend>rclpy</depend>                <!-- ROS Python client -->
  <depend>nav_msgs</depend>             <!-- OccupancyGrid, Path -->
  <depend>geometry_msgs</depend>        <!-- PoseStamped -->
  <depend>av_planner_interfaces</depend>  <!-- our custom msgs -->

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

Listing 27: av_planner/package.xml (key parts)

### 10.4 setup.py: Python Entry Points

The `entry_points` section tells ROS 2 which functions to expose as executable commands:

```
entry_points={
    'console_scripts': [
        'planner_node = av_planner.nodes.planner_node:main',
        'mock_map_node = av_planner.nodes.mock_map_node:main',
    ],
},
```

Listing 28: setup.py entry points

This means `ros2 run av_planner planner_node` calls the `main()` function in `av_planner/nodes/pl`

### 10.5 Building

```
cd ros2_ws
source /opt/ros/humble/setup.bash    # load ROS 2 base
colcon build --symlink-install        # build all packages
source install/setup.bash             # load your packages
```

Listing 29: Build and source

`colcon build` does two things:

1. For `av_planner_interfaces`: runs the `rosidl` code generator to produce Python classes from `.msg` and `.action` files.
2. For `av_planner`: installs the Python package and registers the entry points.

`--symlink-install` creates symlinks instead of copies, so you can edit Python files without rebuilding (you still need to rebuild if you change `.msg` or `.action` files).

## 11 Launch Files: Starting Multiple Nodes

A **launch file** starts multiple nodes at once with their configuration. Your launch file is in XML format:

```
<launch>
  <arg name="params_file"
       default="$(find-pkg-share␣av_planner)/config/params.yaml"/
          >
  <arg name="use_mock_map" default="true"/>

  <!-- Only start mock map if use_mock_map is true -->
  <group if="$(var␣use_mock_map)">
    <node pkg="av_planner" exec="mock_map_node"
          name="mock_map_node" output="screen">
      <param from="$(var␣params_file)"/>
    </node>
  </group>

  <!-- Always start the planner -->
  <node pkg="av_planner" exec="planner_node"
        name="planner_node" output="screen">
```

```
    <param from="$(var␣params_file)"/>
  </node>
</launch>
```

Listing 30: launch/planner.launch.xml

### 11.1 Key Elements

`<arg>`
: Defines a launch argument. `use_mock_map` defaults to `true` but can be overridden:

```
ros2 launch av_planner planner.launch.xml use_mock_map:=false
```

`<group if=...>`
: Conditionally includes nodes. When connecting to a real SLAM system, you set `use_mock_map:=false` to skip the mock.

`<node>`
: Starts a node. `pkg` = package name, `exec` = entry point name, `name` = ROS node name.

`<param from=...>`
: Loads parameters from a YAML file. `$(find-pkg-share av_planner)` resolves to the package's `share/` directory where `params.yaml` was installed.

## 12 How Core Imports Work

The core algorithm files (`src/hybrid_astar/`) are *not* inside the ROS 2 package. They sit in the project root:

```
hybrid_astar_planner/
  src/                        # Core algorithm (NOT a ROS package)
    state.py
    map_handler.py
    hybrid_astar/
      hybrid_astar.py
      motion_model.py
      heuristic.py
      reeds_shepp.py
  ros2_ws/src/                # ROS 2 packages (middleware only)
    av_planner/...
```

The planner node imports these at configuration time using a `project_root` parameter:

```
def _setup_core_imports(project_root: str):
    """Add project root to sys.path so 'from src.*' imports
       resolve."""
    if project_root not in sys.path:
        sys.path.insert(0, project_root)

    from src.state import State as CoreState
    from src.hybrid_astar.motion_model import MotionModel,
       VehicleFootprint
```

```
    from src.hybrid_astar.heuristic import HeuristicCalculator
    from src.hybrid_astar.hybrid_astar import HybridAStar

    return CoreState, MotionModel, VehicleFootprint,
        HeuristicCalculator, HybridAStar
```

Listing 31: Dynamic core imports (planner_node.py)

This design means:

- The core algorithm has **zero ROS dependencies** — it can run standalone via `main.py`
- The ROS node is a **thin wrapper** — it just translates data formats and manages communication
- If you improve the algorithm, you don't need to touch the ROS code
- If you change the ROS interface, you don't need to touch the algorithm

# 13 Putting It All Together: Step by Step

Here is the complete sequence to run the planner:

## 13.1 Build

```
cd ~/Documents/hybrid_astar_planner/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

## 13.2 Launch

```
ros2 launch av_planner planner.launch.xml
```

This starts `mock_map_node` and `planner_node`. The mock node immediately publishes the map. The planner node starts in "unconfigured" state.

## 13.3 Configure and Activate

```
ros2 lifecycle set /planner_node configure
# -> Imports core classes, creates subscribers/publishers/action
   server
# -> Receives map from /map, builds planner pipeline

ros2 lifecycle set /planner_node activate
# -> "Activated. Ready for planning requests."
```

### 13.4 Plan

```
ros2 action send_goal /plan_path av_planner_interfaces/action/
   PlanPath \
  "{start: {pose: {position: {x: 10.0, y: 50.0}, \
           orientation: {w: 1.0}}}, \
    goal: {pose: {position: {x: 90.0, y: 50.0}, \
           orientation: {w: 1.0}}}}"
```

### 13.5 Verify

```
ros2 topic echo /planned_path --once      # see the path
ros2 topic echo /planning_stats --once    # see the metrics
```

### 13.6 Useful Debugging Commands

```
ros2 node list                # list all running nodes
ros2 topic list               # list all active topics
ros2 topic info /map          # show publishers/subscribers
ros2 param list /planner_node  # list all parameters
ros2 param get /planner_node vehicle.wheel_base  # read a param
ros2 lifecycle get /planner_node  # current lifecycle state
```

## 14 Integration with Real Systems

The entire ROS 2 package is designed so that replacing the mock with real components requires **zero code changes**:

| Component | Standalone (testing) | Real System |
|---|---|---|
| Map source | `mock_map_node` (PNG) | SLAM / `map_server` |
| Planning input | CLI action goal | Mission planner node |
| Path consumer | `ros2 topic echo` | Controller node |

To switch to a real system:

```
# Don't start mock map - real SLAM publishes on /map
ros2 launch av_planner planner.launch.xml use_mock_map:=false
```

As long as your SLAM system publishes `nav_msgs/OccupancyGrid` on `/map`, and your mission planner sends goals via the `/plan_path` action, the planner works identically.

# 15 Summary of Key Concepts

| Concept | In Your Project |
|---|---|
| **Node** | `planner_node`, `mock_map_node` — independent programs |
| **Topic** | `/map`, `/planned_path`, `/planning_stats` — named channels |
| **Message** | `OccupancyGrid`, `Path`, `PlanningStats` — structured data |
| **Action** | `/plan_path` — request/feedback/result for planning |
| **Lifecycle** | Unconfigured → Inactive → Active — controlled startup |
| **QoS** | TRANSIENT_LOCAL for map, RELIABLE for everything |
| **Parameters** | `params.yaml` — all algorithm settings |
| **Adapter** | `MapAdapter`, `converters.py` — translate ROS ↔ algorithm |
| **Launch** | `planner.launch.xml` — starts everything with one command |
| **Callback groups** | Separate groups for map and planning — concurrent execution |
| **Executor** | `MultiThreadedExecutor` — enables concurrency |

**The Big Picture**

Your Hybrid A* algorithm is **100% pure Python** with zero ROS imports. The ROS 2 package is a **thin middleware layer** that:

1. Receives map data from other modules (via topics)
2. Translates it into the format your algorithm expects (via adapters)
3. Runs your algorithm (one function call: `planner.plan()`)
4. Translates the result back to ROS messages (via adapters)
5. Publishes the result for other modules to consume (via topics)

That's it. ROS 2 is the plumbing; your algorithm is the engine.